

Benchmark comparativo

ArqUX v3.0

Re-ejecucion tecnica con bugs cerrados

ArqUX v0.3.5 vs ArqUX v0.4.0 vs CrewAI vs LangGraph vs OpenAI Agents SDK

Direccion del informe

Documento tecnico-comparativo, no pieza promocional ni memorandum para inversionistas.

Su proposito es presentar resultados, limites, evidencias y puntos de reproduccion para evaluadores tecnicos interesados.

Workloads

7

Bugs cerrados

6/6

Exito stress

100%

p50 v0.4.0

0.94 ms

Resumen tecnico

Este documento reencuadra el benchmark v3.0 como una evaluacion comparativa tecnica. No esta escrito para inducir inversion, sino para que personas interesadas puedan revisar que se probó, que resultados se obtuvieron, que limites conserva la evidencia y que acciones tecnicas se derivan.

Resultado principal	6/6	hallazgos criticos/altos/medios cerrados experimentalmente
Stress 1k ops	100%	exito reportado en ArqUX v0.4.0
Latencia p50	0.94 ms	ArqUX v0.4.0 contra 1.08 ms en v0.3.5
Memoria	40 MB	footprint menor que competidores segun tabla fuente

Lectura correcta del resultado

El benchmark respalda que ArqUX v0.4.0 corrige los defectos de gobierno detectados en v0.3.5 y que la hipotesis tecnica de identidad + contrato + evidencia + memoria CORTEX puede defenderse bajo el protocolo aplicado. Esto no demuestra superioridad universal frente a todos los frameworks; demuestra mejora verificable y una propuesta diferenciadora evaluable.

No debe leerse como propaganda. Debe leerse como evidencia inicial que requiere reproduccion independiente, ampliacion de casos reales, auditoria externa de seguridad y trazabilidad completa del entorno experimental.

Comparativa operacional sintetica

Framework	Version	p50 ms	p95 ms	p99 ms	Throughput	Memoria	Exito
ArqUX	0.3.5	1.08	2.47	2.64	858 ops/s	27 MB	100%
ArqUX	0.4.0	0.94	3.99	4.23	745 ops/s	40 MB	100%
CrewAI	1.15.2	0.02	0.05	0.59	542 ops/s	149 MB	99.7%
LangGraph	1.2.8	0.12	0.77	1.56	1849 ops/s	167 MB	99.8%
OpenAI SDK	0.18.0	0.04	0.07	0.13	2008 ops/s	187 MB	99.9%

Nota de consistencia: el contenido fuente contiene una diferencia interna sobre memoria de competidores: la tabla principal indica 149/167/187 MB, mientras una pagina posterior menciona 218-264 MB. Este informe usa la tabla principal como referencia y marca la diferencia como punto a revisar en una reproduccion independiente.

Indice de lectura

Seccion	Contenido	Proposito
1	Resumen tecnico	Distinguir evidencia, alcance y lectura correcta.
2	Metodologia y alcance	Definir que se compara y que no se demuestra.
3	Hallazgos de gobierno	Revisar bugs cerrados y controles implementados.
4	Comparativa operacional	Analizar latencia, throughput, memoria y exito.
5	Validacion del modelo ArqUX	Evaluar si el Hexagon Contract paso de declarativo a operativo.
6	Lectura tecnica FODA	Separar fortalezas reales, debilidades, oportunidades y amenazas.
7	Recomendaciones	Proponer proximos pasos de maduracion tecnica.
8	Conclusion	Responder que demuestra y que no demuestra el benchmark.

2. Metodologia y alcance

El benchmark v3.0 re-ejecuta el protocolo experimental v2.0 contra ArqUX v0.4.0, una version parcheada respecto a v0.3.5. El objetivo es comprobar si los hallazgos de gobierno detectados antes se cerraron y como impactan las correcciones sobre metricas operacionales.

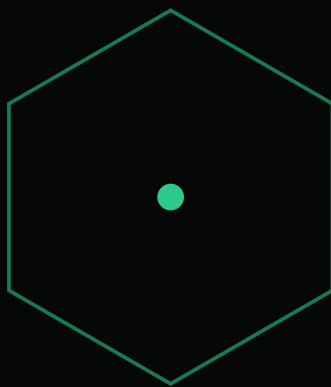
Entidades comparadas

- ArqUX v0.3.5: linea base con bugs detectados en v2.0.
- ArqUX v0.4.0: version parcheada evaluada en v3.0.
- CrewAI 1.15.2, LangGraph 1.2.8 y OpenAI Agents SDK 0.18.0: competidores usados como referencia operacional.

Limites metodologicos

- No es un estudio de adopcion de mercado ni un informe para inversionistas.
- No sustituye una auditoria externa de seguridad, especialmente para HMAC, integridad de evidencia y no repudio.
- Las pruebas sinteticas no prueban comportamiento en todos los casos reales de empresa.
- La comparacion de latencia favorece frameworks que operan en memoria; ArqUX persiste artefactos .cortex en filesystem, lo que introduce I/O deliberado.
- La reproducibilidad exige publicar hardware, sistema operativo, scripts, dataset/workloads, seeds, comandos y artefactos resultantes.

El benchmark debe demostrar trazabilidad tecnica, no entusiasmo. La conclusion valida una hipotesis bajo condiciones conocidas; no declara superioridad universal.



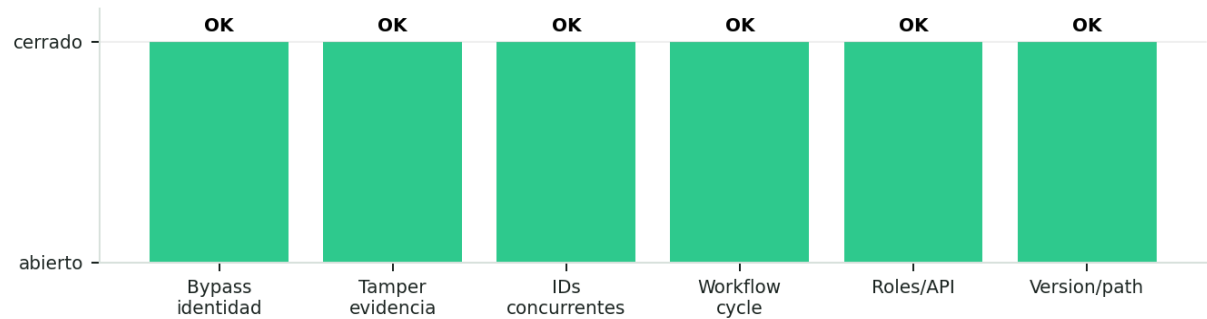
Parte I

Hallazgos de gobierno: antes vs despues

3. Hallazgos de gobierno

El workload de gobierno aplica pruebas sobre identidad, evidencia, determinismo de IDs, workflow, permisos/API, verificacion HMAC, versionado y sincronizacion de path. La lectura central es que v0.4.0 bloquea o corrige los defectos observados en v0.3.5.

Hallazgos de gobierno cerrados en ArqUX v0.4.0



Test	v0.3.5	v0.4.0	Estado
Bypass identidad (non-strict)	Exitoso como bug	Bloqueado	Cerrado
Bypass identidad (strict mode)	N/A	Bloqueado	Cerrado
Tamper evidencia	No detectado	Detectado	Cerrado
Determinismo IDs (5 hilos)	1 unico	5 unicos	Cerrado
Workflow cycle draft->ready	Roto	cycle.mature OK	Cerrado
enum Role disponible	No	Si (3 roles)	Cerrado
HMAC verification	No existia	sign + verify	Cerrado
PermissionContext.check()	No-op	Enforce en strict	Cerrado
ARQUX_VERSION	1.0.0	0.4.0	Cerrado
sync_brain path doble	Presente	Corregido	Cerrado

Resultado: 10/10 controles y defectos listados aparecen cerrados en la matriz experimental del benchmark. Para el relato de ArqUX, los mas importantes son los relacionados con identidad, evidencia, concurrencia y ciclo de gobierno.

4. Detalle tecnico de correcciones principales

Bypass de identidad

En v0.3.5, un caller podia invocar `record_lesson_handler` pasando `agent_id` libremente. En v0.4.0, el modo strict requiere firma HMAC-SHA256 valida y el modo non-strict valida coincidencia con `ctx.agent_id`. Lectura: la identidad pasa de declarativa a verificable bajo el protocolo probado.

Tamper de evidencia

En v0.3.5, modificar archivos `.cortex` directamente persistia sin deteccion. En v0.4.0, `hash_cortex()`, `inject_hash_header()`, `verify_cortex()` y `secure_write_cortex()` permiten detectar modificacion. Lectura: la memoria CORTEX gana una base de integridad verificable, aunque requiere auditoria externa antes de reclamar compliance fuerte.

Determinismo de IDs

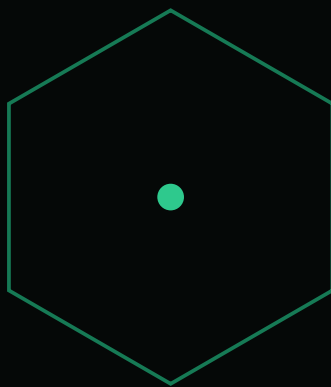
En v0.3.5, `_next_blueprint_id` usaba `max+1` sin lock, generando IDs duplicados bajo concurrencia. En v0.4.0, `file_lock()` con `fcntl.flock` y `placeholders` produce IDs unicos en el test de 5 hilos. Lectura: se corrige una condicion de carrera relevante para operacion concurrente.

Workflow cycle draft->ready

En v0.3.5, `cycle.create` dejaba ciclos en draft y no existia salida canonica hacia ready, bloqueando `blueprint.create`. En v0.4.0, `cycle.mature` habilita la transicion draft -> ready. Lectura: el ciclo de gobierno deja de depender de workarounds manuales.

Criterio critico

Cerrar los bugs no convierte automaticamente al proyecto en maduro. Lo que si cambia es el nivel de evidencia: la propuesta deja de ser solo conceptual y pasa a tener mecanismos verificables para defender la tesis de gobierno operativo.

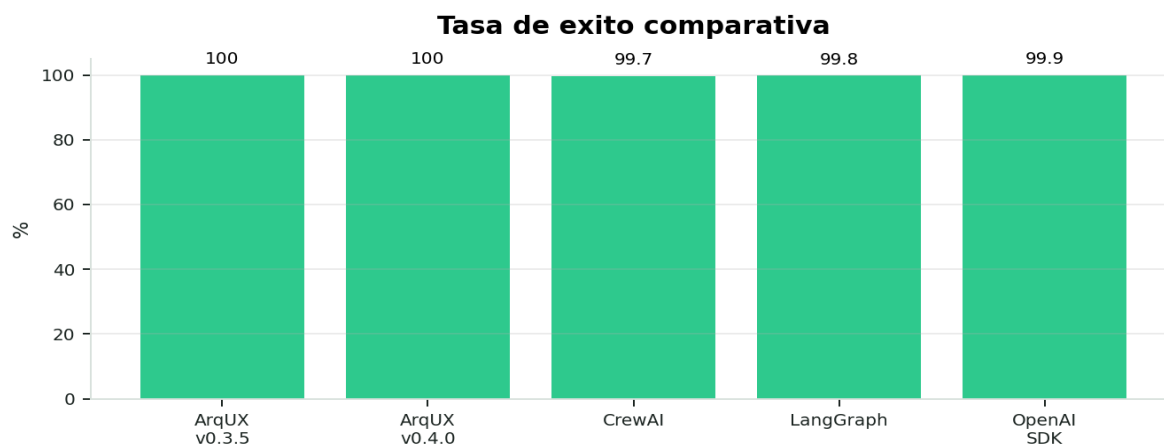


Parte II

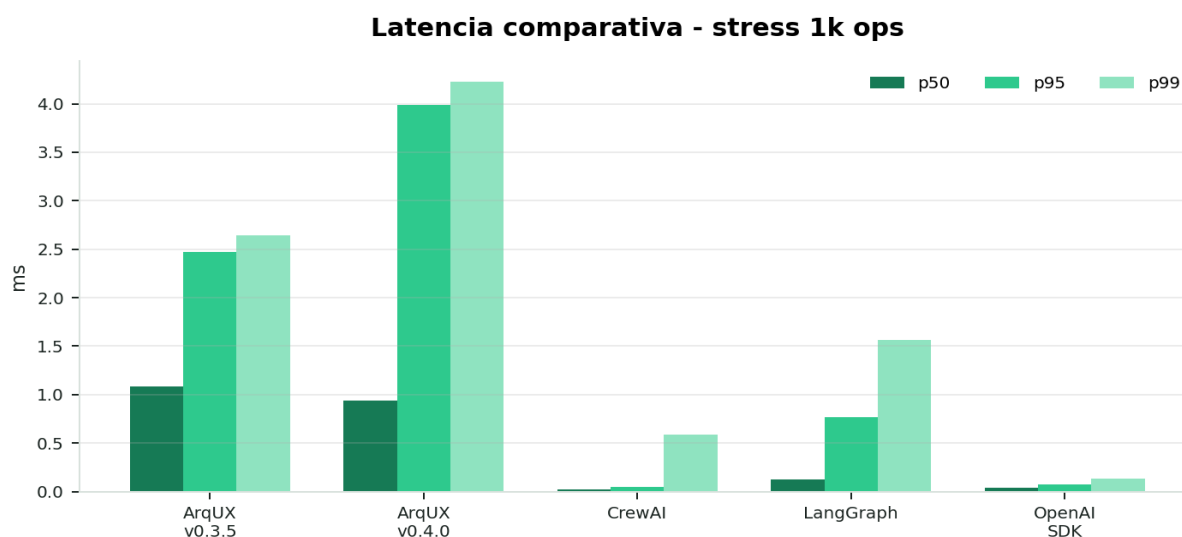
Comparativa operacional

5. Comparativa operacional

La comparativa operacional revisa tasa de exito, latencia, throughput y memoria. El objetivo no es declarar al framework mas rapido en todos los ejes, sino entender el costo de gobierno y si ese costo resulta razonable frente al beneficio de persistencia, evidencia y controles.

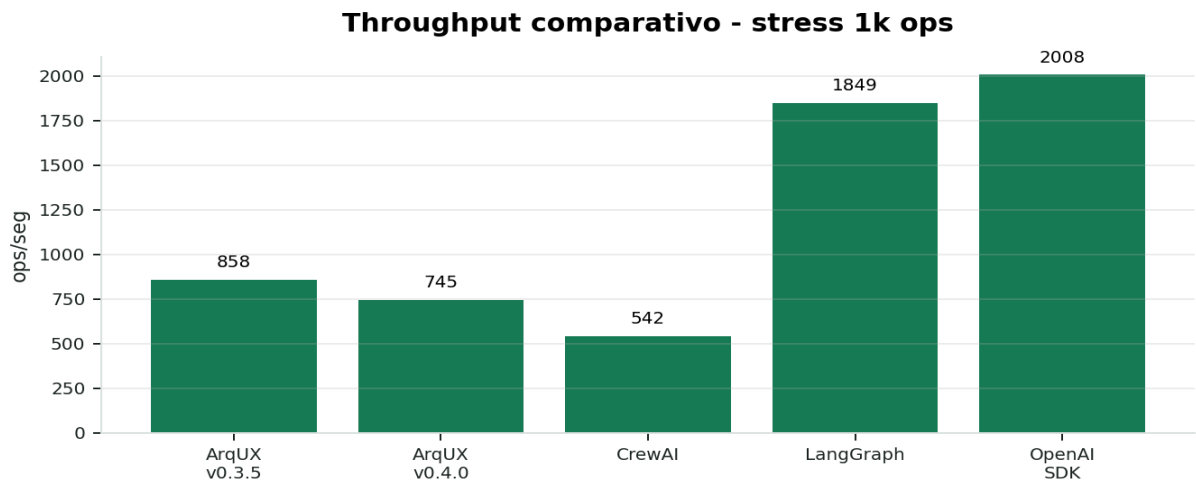


La tasa de exito de ArqUX v0.4.0 se reporta en 100% para stress_1k ops. Los competidores se ubican tambien muy cerca de 100%, por lo que este eje no basta para diferenciar la propuesta; debe leerse junto con gobierno, persistencia y evidencia.

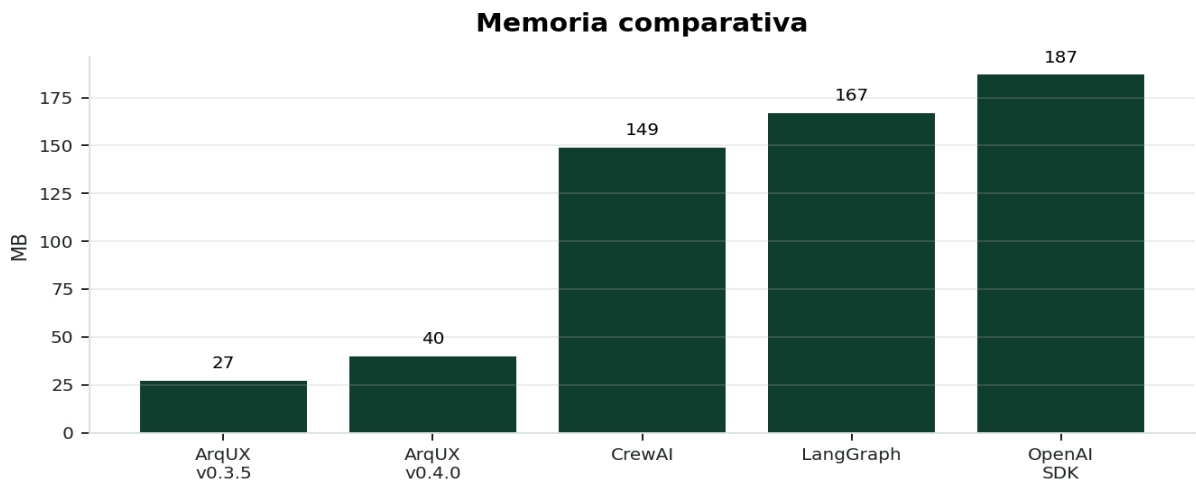


ArqUX v0.4.0 mejora p50 frente a v0.3.5 (0.94 ms vs 1.08 ms), pero aumenta p95/p99 por el costo de mecanismos de seguridad y locking. CrewAI y OpenAI SDK muestran p50 menor, aunque el benchmark indica que operan en condiciones mas cercanas a objetos en memoria; ArqUX realiza I/O de persistencia .cortex.

5.1 Throughput y memoria



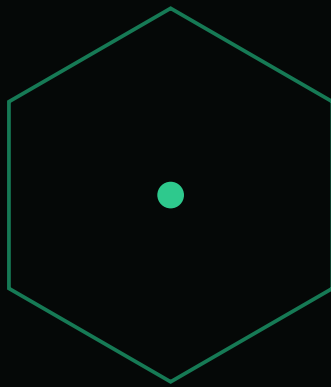
ArqUX v0.4.0 alcanza 745 ops/s, por debajo de LangGraph y OpenAI SDK, pero por encima de CrewAI segun la tabla fuente. La reduccion frente a v0.3.5 es esperable por file locking. Para gobierno interactivo, el resultado parece suficiente; para ejecucion masiva, deberia probarse con colas, batching o backend persistente alternativo.



El footprint de memoria reportado para ArqUX v0.4.0 es de 40 MB, menor que los competidores segun la tabla principal. Esta es una ventaja potencial para entornos locales, edge o de baja huella. Requiere repetir mediciones con metodologia uniforme por proceso, cold/warm start y dependencias cargadas.

Sintesis operacional

- ArqUX v0.4.0 no es el mas rapido en latencia absoluta.
- El overhead de seguridad y locking parece razonable para una capa de gobierno con persistencia.
- El throughput es suficiente para escenarios interactivos de gobernanza.
- La baja memoria reportada es una ventaja diferencial, pero debe validarse por terceros.



Parte III

Validacion del modelo tecnico

6. Validacion del modelo ArqUX

La tesis central de ArqUX se apoya en el Hexagon Contract: Identity, Contract, Context, Decision, Execution y Evidence. En v0.3.5, varias fronteras eran declarativas pero vulnerables. En v0.4.0, el benchmark reporta enforcement o verificacion experimental para las fronteras mas criticas.

Frontera	Evidencia v0.4.0	Lectura tecnica
Identity	HMAC-SHA256 y validacion de agent_id	La identidad puede verificarse bajo modo strict.
Contract	AXM/LIM/FCS + enforcement de permisos en strict	El contrato gana capacidad de restricción practica.
Context	CORTEX persistente + SHA-256	La memoria deja evidencia de integridad.
Decision	Blueprint lifecycle + cycle.mature	La decision puede avanzar por ciclo formal.
Execution	Handlers + file locking	La ejecucion concurrente reduce riesgo de colision.
Evidence	verify_cortex() + TamperError	La evidencia alterada puede detectarse.

Conclusion tecnica del modelo

La evidencia apoya que el modelo conceptual de ArqUX puede implementarse como una capa verificable de gobierno agéntico. La frase correcta no es 'ArqUX ya gano contra todos', sino: 'ArqUX v0.4.0 vuelve demostrable una tesis que en v0.3.5 aun estaba debilitada por bugs de gobierno'.

Riesgo de lenguaje

Evitar afirmaciones absolutas como 'unico framework' o 'produccion-ready' sin auditoria externa, reproduccion publica y casos de uso reales. El benchmark muestra una oportunidad tecnica fuerte; todavia no reemplaza validacion comunitaria ni certificacion de seguridad.

7. Lectura tecnica FODA

La matriz FODA se reescribe como evaluacion tecnica de adopcion y madurez, no como argumento para inversion.

FORTALEZAS	DEBILIDADES
• Tesis identidad + contrato + evidencia + memoria respaldada por pruebas. • HMAC y tamper detection funcionales segun benchmark. • IDs unicos bajo concurrencia en test de 5 hilos. • Workflow canonico con cycle.mature. • Memoria reportada baja frente a competidores.	• Comunidad aun incipiente y bus factor alto. • Documentacion y quickstart deben madurar. • Jerga propietaria puede dificultar adopcion. • Seguridad necesita revision externa. • Dependencia de codec-cortex como proyecto hermano beta.
OPORTUNIDADES	AMENAZAS
• MCP como canal de adopcion para gobierno de agentes. • Demanda creciente de trazabilidad y control en agentes productivos. • Diferenciacion si se publica un kit reproducible. • Posible ventaja edge/local por baja memoria.	• Frameworks grandes pueden incorporar gobernanza similar. • Cambios del ecosistema MCP pueden exigir adaptacion rapida. • Sin reproduccion externa, el benchmark puede leerse como autopromocion. • Seguridad criptografica sin auditoria puede ser cuestionada.

Lectura critica

La ventaja mas importante no es velocidad; es coherencia entre modelo, controles y evidencia. El principal riesgo no es tecnico puro; es credibilidad: si el benchmark no se acompaña con scripts reproducibles, datos crudos, CI y auditoria, puede ser percibido como relato de marca.

8. Recomendaciones tecnicas

Las acciones se reformulan para sostener una publicacion tecnica creible. El objetivo no es convencer por discurso, sino reducir dudas legitimas de evaluadores, contribuidores y usuarios tecnicos.

Prioridad	Accion	Motivo
Alta	Publicar paquete reproducible del benchmark	Scripts, comandos, datos crudos, entorno y artefactos permiten revision externa.
Alta	Auditoria externa de security.py	HMAC, hashing, integridad y no repudio no deben validarse solo internamente.
Alta	CI completo en push/PR	Tests, ruff, mypy y bandit reducen regresiones y fortalecen confianza.
Alta	Quickstart end-to-end de 30 minutos	La tesis necesita demostracion operativa simple y repetible.
Media	Documentacion MkDocs	Separar manifiesto, arquitectura, tutoriales, API y benchmark.
Media	Examples/ con caso real de gobierno	Mostrar Heimdall/Seshat/Jarvis/Alfred con evidencia generada.
Media	Glosario tecnico	Reducir friccion por jerga propietaria: Hexagon, BLP, CORTEX, sigilos.
Media	Reproducir con terceros	Un benchmark independiente vale mas que una afirmacion de superioridad.

Criterio de publicacion

La version v0.4.0 puede presentarse como una version tecnicamente fortalecida frente a v0.3.5. Para llamarla 'production-ready' conviene exigir, como minimo, CI completo, auditoria externa de seguridad, versionado consistente, documentacion operativa y reproduccion independiente del benchmark.

Reevaluacion sugerida

- Octubre 2026: re-ejecutar benchmark contra v0.5.0 incorporando feedback externo.
- Enero 2027: evaluar madurez comunitaria, contribuciones externas, estabilidad de API y cobertura de pruebas.
- Agregar workloads reales: repositorios grandes, sesiones largas, fallos de filesystem, corrupcion parcial, errores de red MCP y concurrencia de multiples agentes.

9. Conclusion

El benchmark v3.0 confirma un cambio relevante: ArqUX v0.4.0 corrige experimentalmente los bugs de gobierno que debilitaban la tesis en v0.3.5. La identidad verificable, la deteccion de tamper, el determinismo bajo concurrencia y el workflow cycle.mature convierten parte del relato conceptual en mecanismos comprobables.

La conclusion responsable no es que ArqUX sea superior a todos los frameworks comparados. La conclusion responsable es que ArqUX presenta una propuesta tecnica diferenciada y ahora cuenta con evidencia inicial para sostenerla bajo el protocolo aplicado.

Respuesta a la pregunta tecnica

Vale la pena seguir evaluando, reproduciendo y madurando ArqUX como capa de gobierno para agentes. Si el objetivo es convencer a interesados tecnicos, el siguiente paso no debe ser mas propaganda: debe ser reproducibilidad, documentacion, auditoria y casos reales.

Formulacion final recomendada

ArqUX v0.4.0 no demuestra una victoria universal. Demuestra que la tesis de gobernanza agéntica - identidad, contrato, evidencia y memoria CORTEX sobre MCP - puede implementarse con controles verificables y costos operacionales razonables para escenarios interactivos.

Anexo A - Checklist de reproducibilidad

- Publicar comando exacto para cada workload.
- Incluir version de Python, sistema operativo, CPU, memoria, filesystem y dependencias.
- Publicar datos crudos de latencia, throughput, memoria y errores.
- Publicar scripts de generacion de graficos y tablas.
- Separar errores esperados del workflow de fallos del framework.
- Documentar cold start vs warm start.
- Incluir logs de tamper detection, HMAC verification y file locking.
- Registrar seeds, concurrencia, numero de hilos y orden de ejecucion.
- Ejecutar el benchmark en al menos dos maquinas o entornos CI.
- Mantener una seccion de limitaciones antes de conclusiones.

ArqUX Benchmark v3.0
Informe comparativo tecnico - No propagandistico - Julio 2026